SERVER
CONTROLLER
GameController
- model: Game
~ numberPlayers: int
- players: List<Player>
+ setPlayerNumber(int): void
+ initializeModel(): void
+ setChosenTiles(List<Tile>): void
+ addChosenCoordinate(int[]): void
+ addChosenTile(int[]): void
+ nextPlayer(): void
+ dropTiles(List<Tile>, int): void
+ setPlayerNickname(String): boolean
+ checkCorrectCoordinates(int[], List<int[]>): boolean
+ calculateWinner(): void
+ setBorderTiles(): void
+ interpretMessage(Message): void
MODEL
Game
- commonGoals: CommonGoalCard[]
- bag: Bag
- propertyChangeSupport: PropertyChangeSupport
- numberPartecipants: int
- players: List<Player>
- lastTurn: boolean
- currentPlayer: Player
- start: boolean
- end: boolean
- board: Board
+ setCurrentPlayer(Player): void
+ isLastTurn(): boolean
+ addPropertyChangeListener(PropertyChangeListener): void
+ setLastTurn(boolean): void
+ findWinner(): void
+ getAvailableTilesForCurrentPlayer(): List<int[]>
+ setGame(int, List<Player>): void
+ getPlayers(): List<Player>
+ getCommonGoals(): CommonGoalCard[]
+ endGame(): void
+ propertyChange(PropertyChangeEvent): void
+ getBoard(): Board
+ setStart(boolean): void
+ setFirstPlayer(): void
+ getCurrentPlayer(): Player
+ setEnd(boolean): void
+ getBag(): Bag
+ removePropertyChangeListener(PropertyChangeListener): void
Bag
- bag: List<Tile>
+ getTile(): Tile
Costants
+ SHELF_COLUMN: int
+ TILES_PER_COLOR: int
+ DECK_SIZE: int
+ COMMON_CARDS_PER_GAME: int
+ END_GAME_POINT: int
+ SHELF_ROWS: int
+ TILES_TYPES: int
+ MAX_TILES_PER_TURN: int
DeckCommon
- deck: List<String>
+ popCommonGoalCard(): String
12
CommonGoalCard
- scores: Stack<Integer>
- description: String
- cardStrategy: CardStrategy
1
- setCardStrategy(DeckCommon): void
+ fillStack(int): void
+ getPoint(): int
+ getDescritpion(): String
+ algorythm(Shelf): boolean
<<interface>>
CardStrategy
+ toString(): String
+ algorythm(Shelf): boolean
Board
- board: Tile[][]
- size: int
+ checkRefill(): boolean
- setupBoard(String): void
- makeBoard(List<Tile>): void
+ getAvailableTiles2(): List<int[]>
+ fillBoard(Bag): void
+ popTile(int, int): Tile
+ getBoard(): Tile[][]
+ getAvailableTiles(): List<Tile>
+ getTile(int, int): Tile
Player
- points: int
- shelf: Shelf
- goalsCompleted: boolean[]
- personalGoalCardCard: PersonalGoalCard
- nickname: String
- playingTurn: boolean
- chosenTiles: List<Tile>
- seat: boolean
+ getNickname(): String
+ setSeat(boolean): void
+ setPrivateCard(PersonalGoalCard): void
+ checkPersonalPoints(): int
+ addPoints(int value): void
+ addPoints(CommonGoalCard): void
+ play(Board, CommonGoalCard[]): void
- selectColumn(): int
+ getShelf(): Shelf
- getTile(Board): void
+ getSeat(): boolean
PersonalGoalCard
~ goalsShelf: Tile[][]
~ cardName: String
12
DeckPersonal
- deck: List<String>
+ popPersonalCard(): PersonalGoalCard
Tile
- color: Color
- type: int
+ getType(): int
+ getColor(): Color
132
22 per type
<<enumeration>>
Color
+ GREEN:
+ YELLOW:
+ BLUE:
+ PINK:
+ WHITE:
+ CYAN:
+ TRANSPARENT:
+ values(): Color[]
+ valueOf(String): Color
Shelf
- shelf: Tile[][]
+ checkRowFulness(int): boolean
+ checkColumnEmptiness(int): int
+ setTile(int, int, Tile): void
+ getTile(int, int): Tile
+ getColumn(int): Tile[]
+ copyShelf(): Shelf
+ putTile(int, int, Tile): void
- countAdjacents(Shelf, int, int, int): int
+ checkAdjacents(): int
+ dropTiles(List<Tile>, int): void
+ isFull(): boolean
+ getRow(int): Tile[]
+ getShelf(): Tile[][]
CommonGoalCard12
+ algorythm(Shelf): boolean
+ toString(): String
CommonGoalCard9
- colorsOccurence: boolean[]
- columnCounter: int
+ algorythm(Shelf): boolean
+ toString(): String
CommonGoalCard7
+ algorythm(Shelf): boolean
+ toString(): String
CommonGoalCard5
+ toString(): String
+ algorythm(Shelf): boolean
CommonGoalCard8
+ toString(): String
+ algorythm(Shelf): boolean
CommonGoalCard4
+ algorythm(Shelf): boolean
+ toString(): String
CommonGoalCard10
- colorsOccurence: boolean[]
- colorBonus: boolean
- rowCounter: int
+ toString(): String
+ algorythm(Shelf): boolean
CommonGoalCard6
+ toString(): String
+ algorythm(Shelf): boolean
CommonGoalCard2
+ toString(): String
+ algorythm(Shelf): boolean
+ getId(): int
CommonGoalCard3
- tmpShelf: Shelf
+ getId(): int
+ toString(): String
- removeRow(int): void
- removeColumn(int): void
CommonGoalCard11
+ algorythm(Shelf): boolean
+ toString(): String
CommonGoalCard1
+ toString(): String
+ algorythm(Shelf): boolean
+ getId(): int
<<interface>>
EventListener
<<interface>>
PropertyChangeListener
+ propertyChange(PropertyChangeEvent): void
ServerImpl
- model: Game
- controller : GameController
- port : int
- ip : String
+ startServer(): void
- sendModelView(): ModelView
- recieveMessages(): Object
+ propertyChange(PropertyChangeEvent): void
- generateModelView(Model): GamelView
<<interface>>
Serializable
GameView
- model: Game
~ serialVersionUID: long
+ isLastTurn(): boolean
+ getAvailableTilesForCurrentPlayer(): List<int[]>
+ getCommonGoals(): CommonGoalCard[]
+ getMaxColumnSpace(): int
+ getCurrentPlayerChosenTiles(): List<Tile>
+ getCurrentPlayerSeat(): boolean
+ getCurrentPlayerNickname(): String
+ getCurrentPlayerShelf(): Shelf
+ getCurrentPlayerPersonalCard(): PersonalGoalCard
+ getCurrentPlayerChosenCoordinates(): List<int[]>
+ getPlayers(): List<Player>
+ getBoard(): Board
+ getCommonGoalCards(): CommonGoalCard[]
+ getCurrentPlayer(): Player
Client
TUI
<<annotation>>
FunctionalInterface
<<interface>>
Runnable
+ run(): void
TextualUI
- controller: GameController
- modelView: GameView
- propertyChangeSupport:PropertyChangeSupport
+ askCoordinates(): int
~ printAvailableTiles(List<int[]>): void
~ printBoard(Board): void
~ printPersonalGoalShelf(PersonalGoalCard): void
+ askNumber(): int
- askNickName(int): void
+ propertyChange(PropertyChangeEvent): void
+ newTurn(GameView): void
- askColumn(): int
+ printShelf(Shelf): void
+ run(): void
+ lastTurn(GameView): void
- askOrder(): void
+ addPropertyChangeListener(PropertyChangeListener): void
<<interface>>
Runnable
+ run(): void
«interface»
PropertyChangeListener
«interface»
EventListener
ClientImpl
- view: TexualUi
- ip : String
- port : int
- serverSocket: ServerSocket
+ run(): void
+ propertyChange(PropertyChangeEvent): void
- sendMessage(Message): void
- receiveModelView(): void
<<interface>>
Serializable
Message
- numberPlayers : int
- nickName: String
- column : int
- coordinates : List<Int[2]>
+ Message(Int numberPlayers): void
+ Message(String nickName): void
+ Message(Int column): void
+ Message(List<Int[2]> coordinates): void
+ getNumberPlayers: int
+ getNickName: String
+ getColumn : int
+ getCoordinates : List<Int[2]>
NETWORK